

Министерство науки и высшего образования Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

ITMO University

ЛАБОРАТОРНАЯ 1

По дисциплине Виртуализация сетевых функций

Тема работы Установка GNS3 и базовая настройка

Обучающийся Фамилия Имя Отчество

Факультет Прикладной информатики

Группа К3422

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в
инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Титов Г. К.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Аминов Н.С.

(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 Установка GNS3	4
2 Настройка GNS3 и проверка работоспособности	5
3 Добавление образов и базовых узлов	8
4 Тестовый проект	11
4.1 Настройка маршрутизатора Cisco 7200	12
4.2 Настройка VPCS	13
4.3 Настройка Ubuntu Docker	13
4.4 Настройка SONiC VS	14
4.5 Проверка сетевой связности	15
4.6 Проверка работоспособности.....	15
5 Заключение	18

ВВЕДЕНИЕ

Цель лабораторной работы — ознакомление с виртуальным лабораторным стендом GNS3, освоение его установки и базовой настройки, а также подготовка рабочей среды для моделирования и исследования компьютерных сетей.

Для достижения поставленной цели были определены следующие задачи:

1. Установить GNS3 на рабочую станцию под управлением Windows 11.
2. Импортировать GNS3 VM в VirtualBox, настроить её ресурсы и подключить к приложению GNS3.
3. Проверить взаимодействие GNS3 с компонентами виртуализации и эмуляции: VirtualBox, QEMU, Dynamips и Docker.
4. Добавить образ SONiC VS, создать шаблон QEMU и настроить параметры запуска виртуального устройства.
5. Подготовить дополнительные узлы лабораторного стенда: маршрутизатор Cisco 7200, хост VPCS и контейнер Ubuntu Docker.
6. Создать тестовую топологию, соединить устройства и настроить IP-адреса интерфейсов и шлюзы.
7. Проверить сетевую связность между узлами с помощью команды `ping`.
8. Подготовить отчёт со скриншотами настроек, созданной топологии и результатов проверки связи.

1 Установка GNS3

Так как для получения установщика с официального ресурса требуется ждать 2 дня, мы установим его с sourceforge.net.

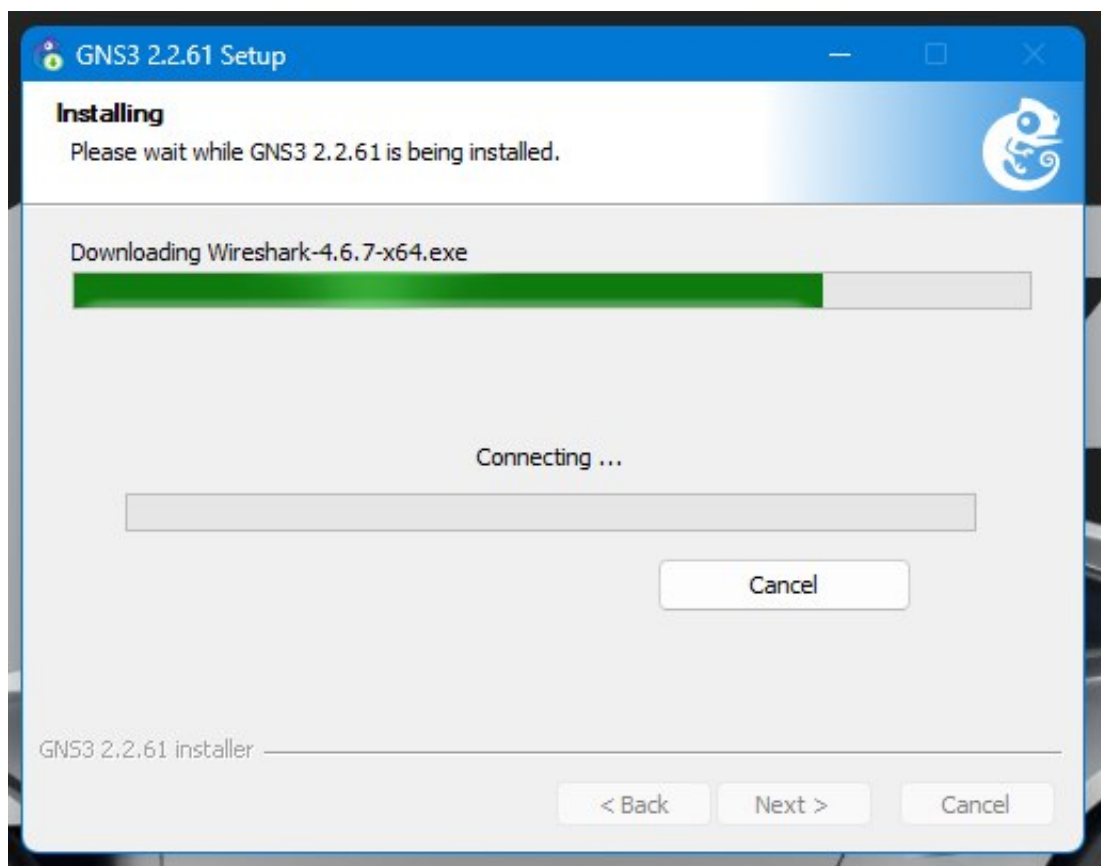


Рисунок 1.1 — Установка GNS3

2 Настройка GNS3 и проверка работоспособности

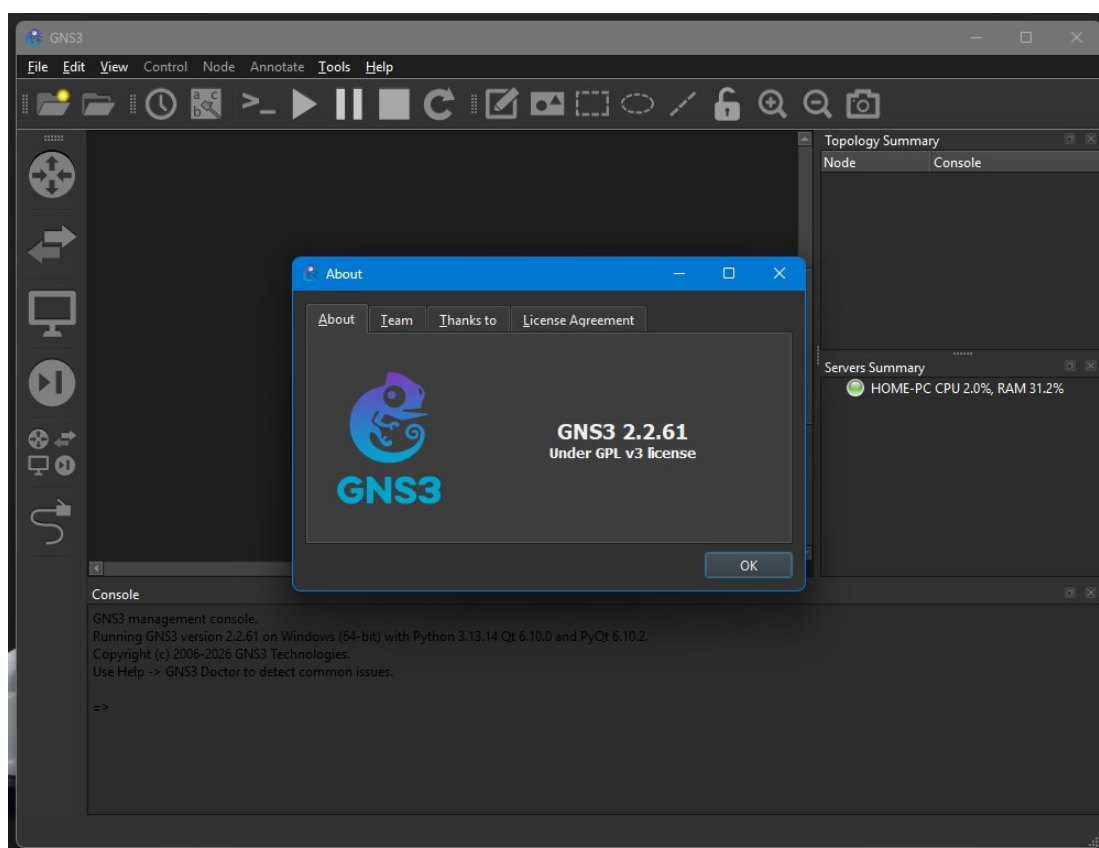


Рисунок 2.1 — Проверяем версию GNS3

Также для выполнения работ мы установим GNS3 VM. В ней мы будем поднимать наши сетевые устройства.

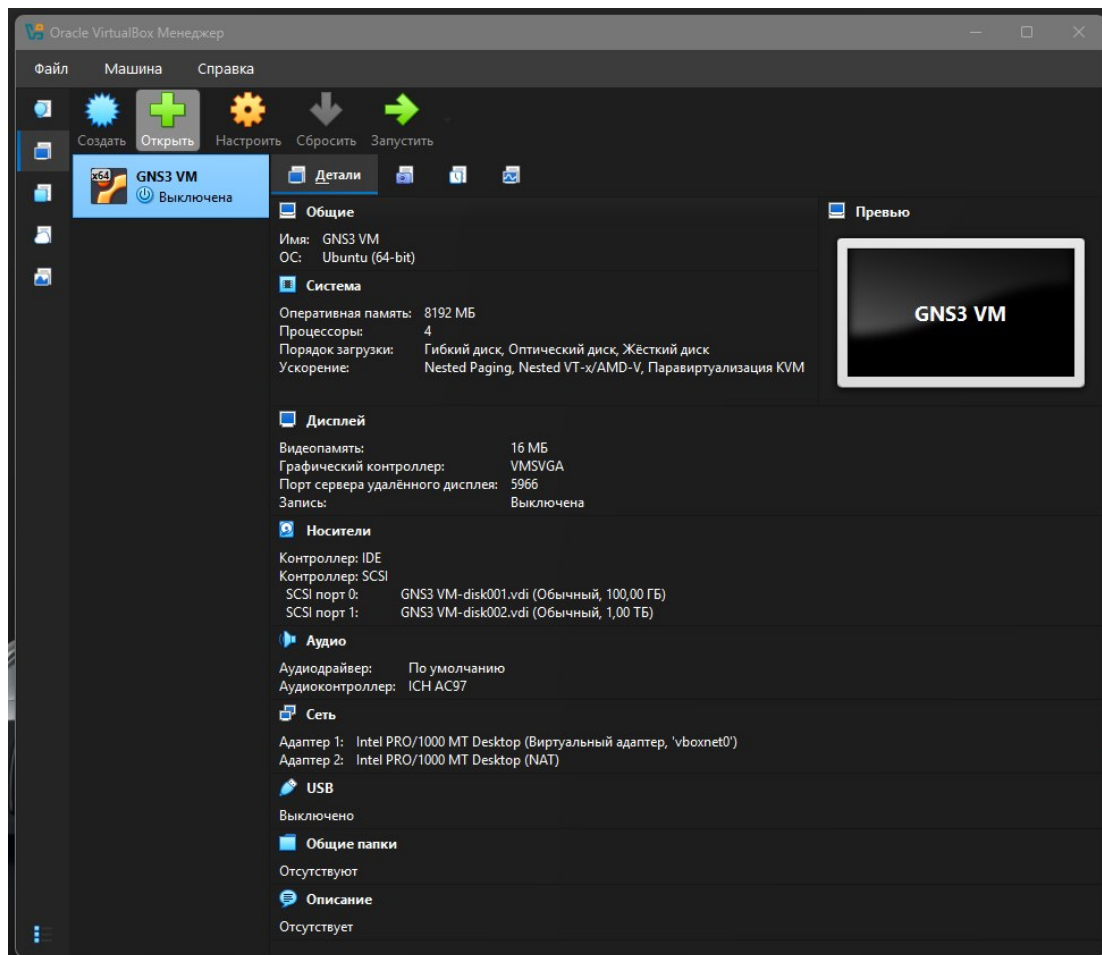


Рисунок 2.2 — Конфигурация GNS3 VM

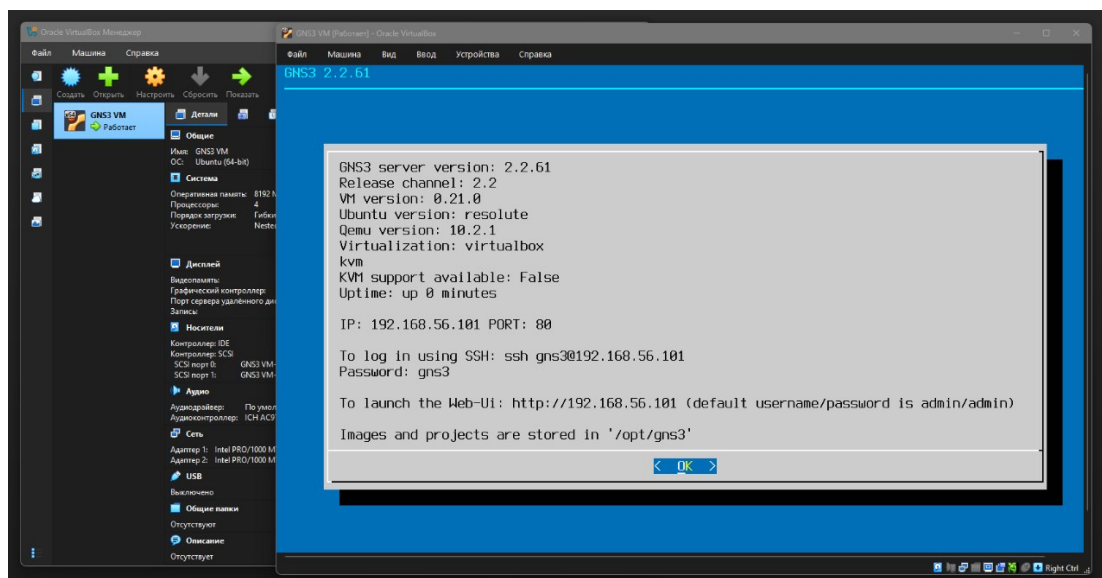


Рисунок 2.3 — Запуск и проверка работоспособности

Также явно указываем в GNS3, что запускать наши проекты мы будем при помощи виртуальной машины GNS3 VM при помощи гипервизора VirtualBox.

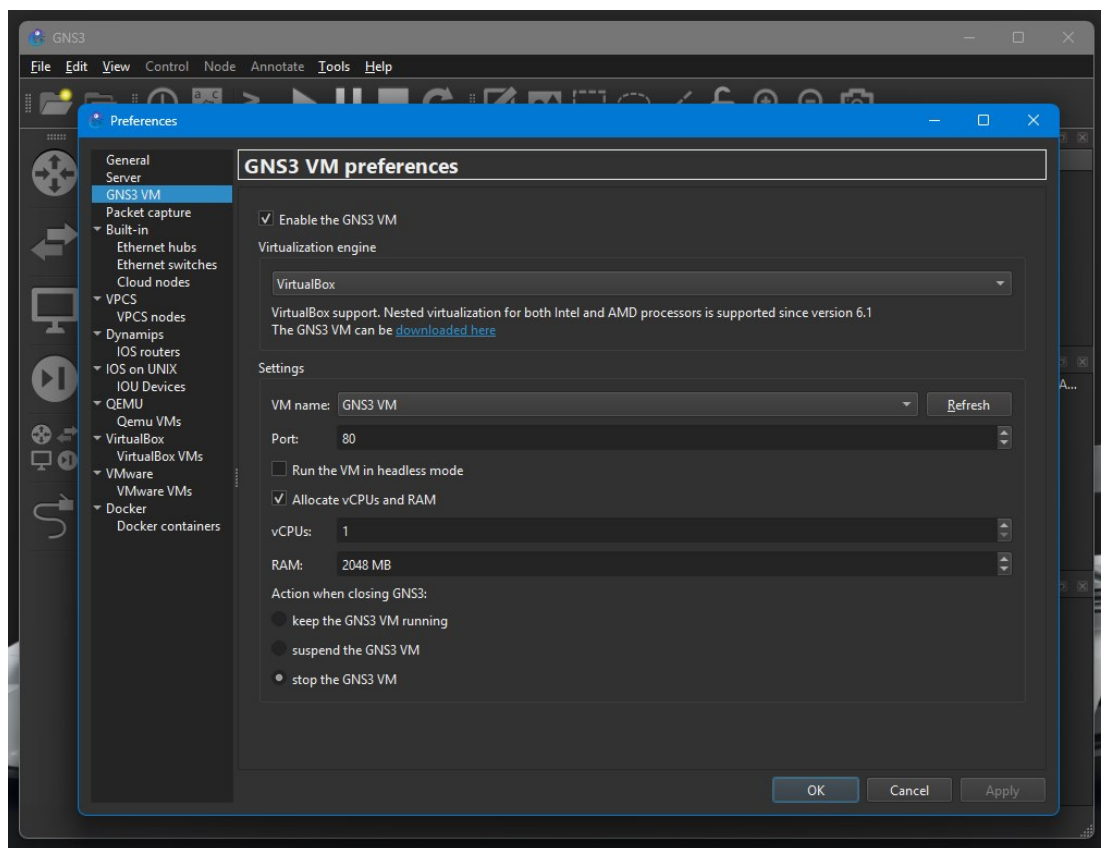


Рисунок 2.4 — Конфигурация GNS3 VM

3 Добавление образов и базовых узлов

Далее добавляем образ SONIC

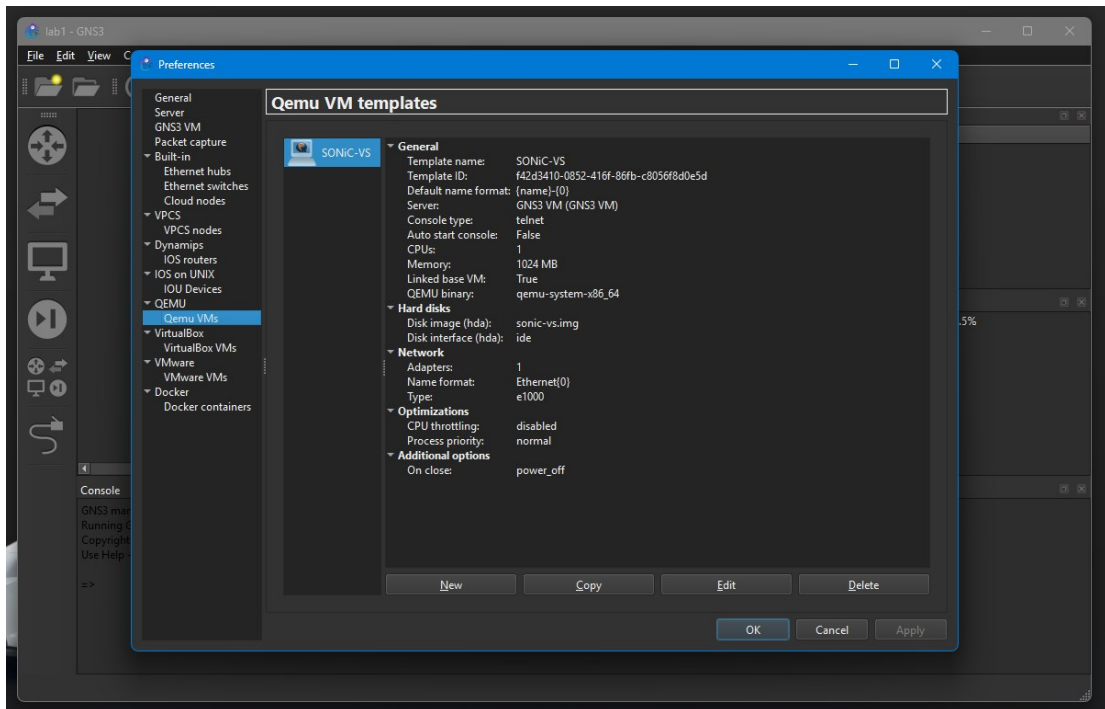


Рисунок 3.1 — Шаблон SONiC-VS

Если вылезает ошибка с kvm, заходим на ВМ и в `gns3_server.conf` дописываем в конец файла блок QEMU:

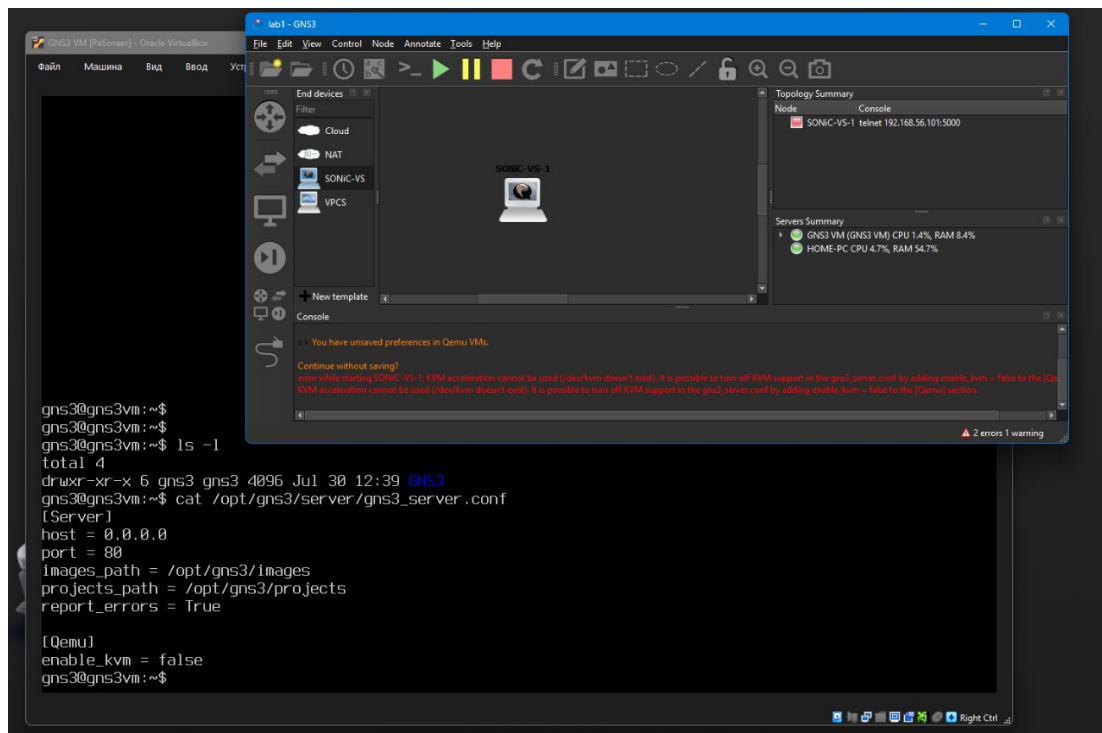


Рисунок 3.2 — Ошибка kvm

После ребутиим наш свитч и проверяем работоспособность путем подключения по консоли:

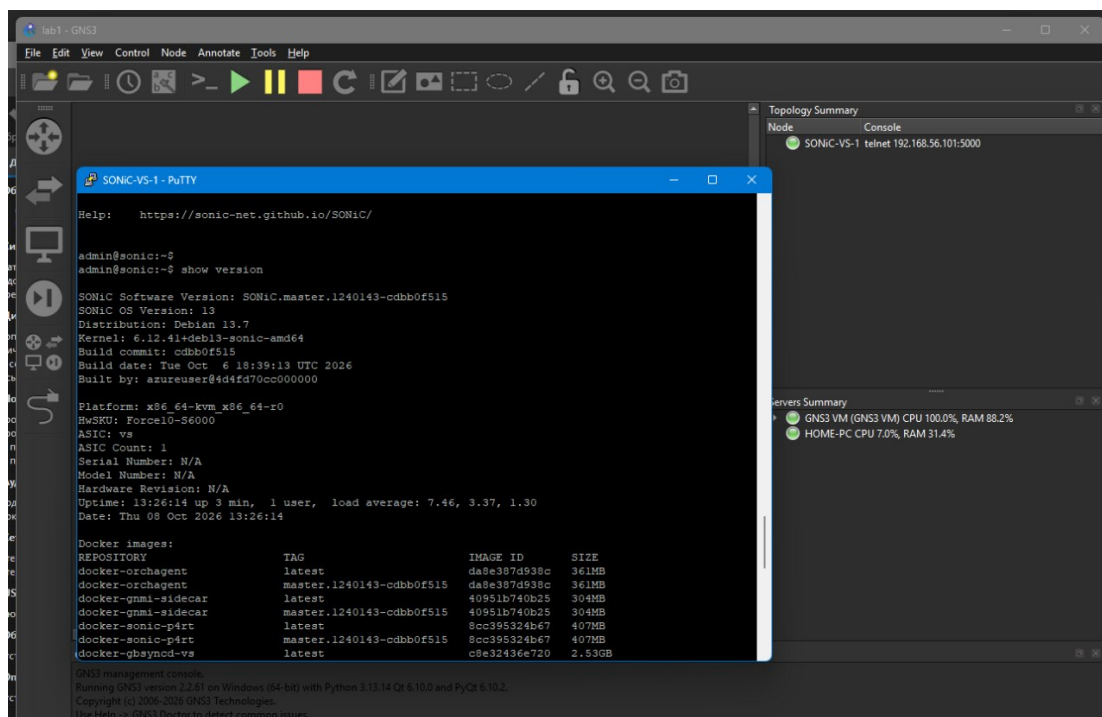


Рисунок 3.3 — Проверка работоспособности SONiC-VS

Далее добавляем роутер Cisco 7200. Импортируем готовый образ и шаблон

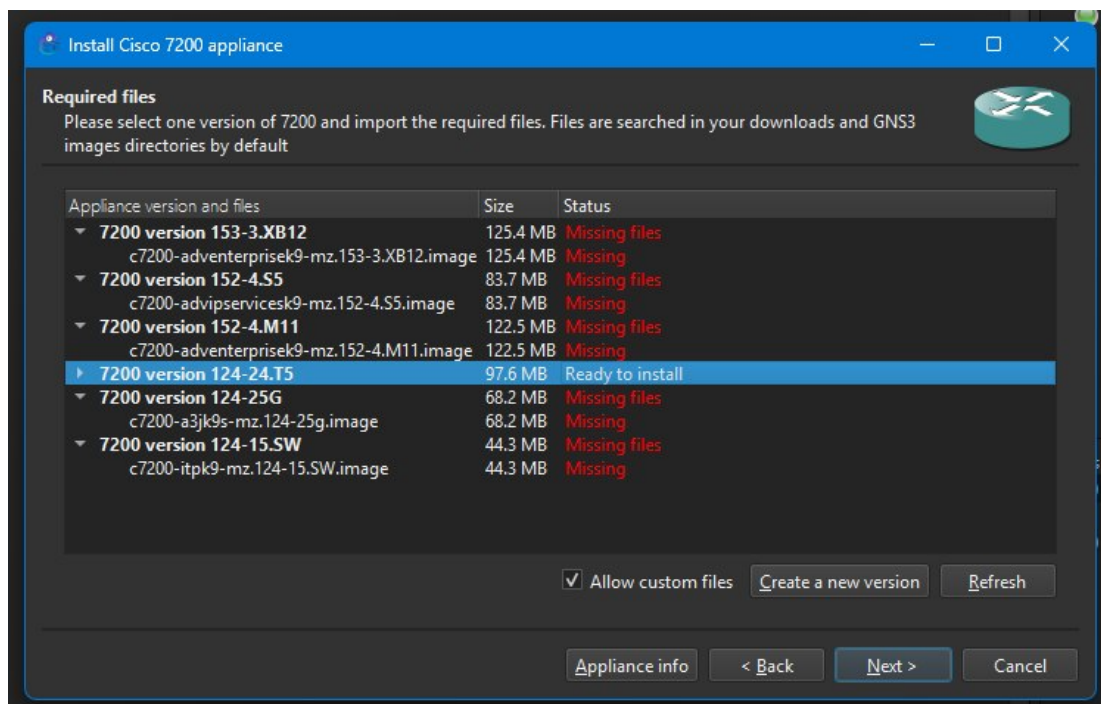


Рисунок 3.4 — Импорт Cisco 7200

4 Тестовый проект

В тестовый проект были добавлены следующие устройства с соответствующими локальными сетями:

1. Ubuntu Docker Guest
2. Cisco 7200.
3. VPCS
4. SONiC-VS

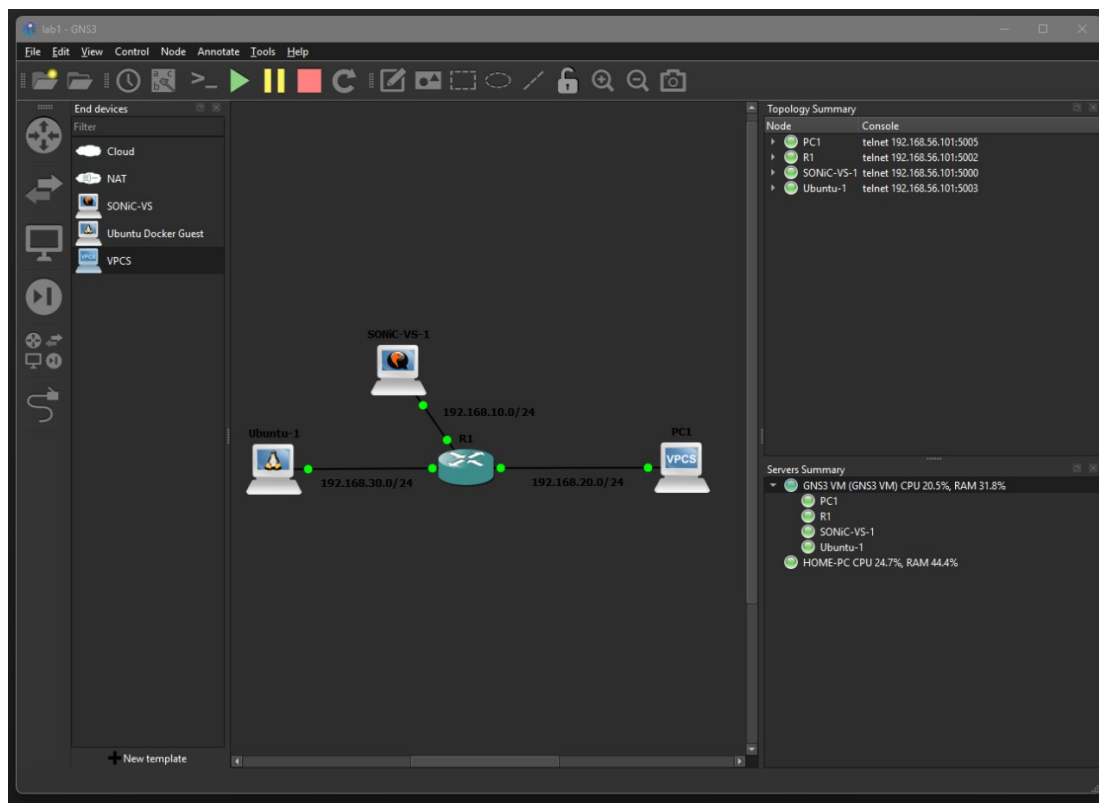


Рисунок 4.1 — Схема тестового проекта

Каждое конечное устройство подключалось к отдельному интерфейсу маршрутизатора и размещалось в собственной подсети.

Для устройств использовалась следующая адресация:

Устройство и интерфейс	IP-адрес	Шлюз
Cisco, FastEthernet0/0	192.168.10.1/24	—
Cisco, FastEthernet1/0	192.168.20.1/24	—
Cisco, FastEthernet2/0	192.168.30.1/24	—
SONiC, eth0	192.168.10.2/24	192.168.10.1
VPCS, Ethernet0	192.168.20.2/24	192.168.20.1
Ubuntu Docker, eth0	192.168.30.2/24	192.168.30.1

4.1 Настройка маршрутизатора Cisco 7200

Для подключения трёх узлов на маршрутизаторе использовались интерфейсы FastEthernet0/0, FastEthernet1/0 и FastEthernet2/0. Каждому интерфейсу был назначен адрес соответствующей подсети. Команда `no shutdown` включала интерфейс.

Настройка выполнялась через консоль Cisco IOS:

```
enable
configure terminal
hostname R1

interface FastEthernet0/0
ip address 192.168.10.1 255.255.255.0
no shutdown
exit

interface FastEthernet1/0
ip address 192.168.20.1 255.255.255.0
no shutdown
exit

interface FastEthernet2/0
ip address 192.168.30.1 255.255.255.0
no shutdown
```

```
exit
```

```
end
```

```
copy running-config startup-config
```

Для проверки состояния интерфейсов и таблицы маршрутизации использовались команды:

```
show ip interface brief
```

```
show ip route
```

Поскольку все три подсети непосредственно подключены к маршрутизатору, добавление статических маршрутов между ними не требовалось.

4.2 Настройка VPCS

На узле VPCS были заданы IP-адрес, маска подсети и шлюз по умолчанию. В качестве шлюза использовался адрес интерфейса FastEthernet1/0 маршрутизатора.

```
ip 192.168.20.2 255.255.255.0 192.168.20.1
```

```
show ip
```

```
save
```

Команда `show ip` использовалась для проверки параметров, а команда `save` — для сохранения конфигурации VPCS.

4.3 Настройка Ubuntu Docker

Сетевые параметры контейнера Ubuntu задавались через редактор сетевой конфигурации узла в GNS3. Для интерфейса `eth0` использовалась статическая конфигурация в файле `/etc/network/interfaces`:

```
auto eth0
```

```
iface eth0 inet static
```

```
address 192.168.30.2
netmask 255.255.255.0
gateway 192.168.30.1
```

После сохранения настроек контейнер запускался с заданными параметрами. Проверка адреса и маршрута по умолчанию выполнялась командами:

```
ip addr show dev eth0
ip route
```

4.4 Настройка SONiC VS

Для подключения SONiC использовался управляющий интерфейс `eth0`. Первоначально адрес и шлюз задавались штатной командой SONiC:

```
sudo config interface ip add eth0 192.168.10.2/24 192.168.10.1
```

Однако проверка через `ip addr` показала, что IPv4-адрес не появился на интерфейсе. Для проверки сетевого соединения адрес и маршрут по умолчанию были временно назначены средствами Linux:

```
sudo ip addr replace 192.168.10.2/24 dev eth0
sudo ip route replace default via 192.168.10.1 dev eth0
```

Применение параметров и доступность маршрутизатора проверялись командами:

```
ip -4 addr show dev eth0
ip route
ping -c 4 192.168.10.1
```

После назначения адреса SONiC успешно обменивался ICMP-пакетами с маршрутизатором. В зафиксированном тесте были получены ответы на все шесть отправленных запросов, потери пакетов составили 0 %.

Дополнительно выполнялась настройка через конфигурационную базу SONiC по примеру отчёта:

```
sudo config interface ip add eth0 192.168.10.2/24
sudo config route add prefix 0.0.0.0/0 nexthop 192.168.10.1
sudo config save -y
```

Наличие записи управляющего интерфейса проверялось командой:

```
sudo sonic-db-cli CONFIG_DB HGETALL \
  'MGMT_INTERFACE|eth0|192.168.10.2/24'
```

В базе появилась запись адреса со служебным полем `NULL`. При этом сохранение полного набора параметров и их автоматическое применение после перезагрузки на данном этапе не были подтверждены. Команды `ip addr` и `ip route` обеспечивали временную настройку текущего состояния Linux.

4.5 Проверка сетевой связности

С маршрутизатора выполнялись проверки доступности конечных устройств:

```
ping 192.168.10.2
ping 192.168.20.2
ping 192.168.30.2
```

Для VPCS первоначальная проверка показала четыре ответа из пяти, для Ubuntu Docker — пять ответов из пяти. SONiC первоначально был недоступен из-за отсутствия IPv4-адреса на интерфейсе `eth0`. После временного назначения адреса была подтверждена связь от SONiC до маршрутизатора.

В данной топологии маршрутизацию между подсетями выполнял Cisco 7200. Проверка SONiC через управляющий интерфейс подтверждала доступность самой операционной системы, но не проверяла коммутацию трафика между её сетевыми портами.

4.6 Проверка работоспособности

Далее на каждом из устройств командой `ping` проверяем доступность других узлов

```
R1#
R1#
R1#
R1#
R1#
R1#ping 192.168.10.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 44/61/88 ms
R1#ping 192.168.20.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 44/47/52 ms
R1#ping 192.168.30.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.30.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 44/56/88 ms
R1#
```

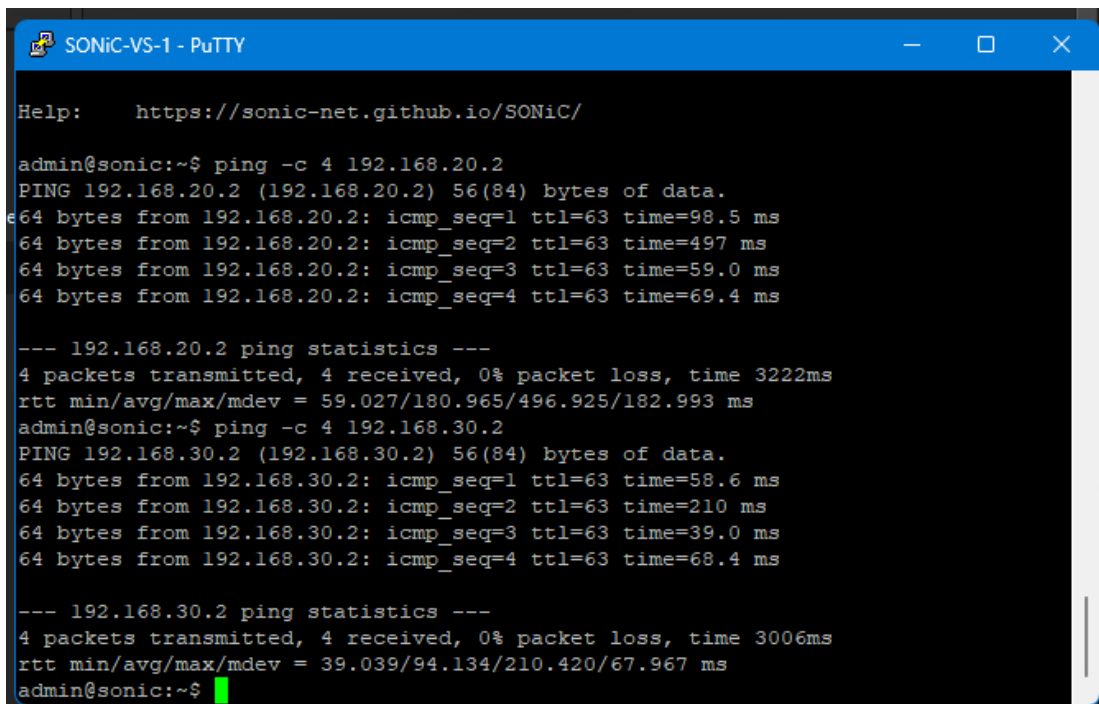
Рисунок 4.2 — Cisco 7200

```
root@Ubuntu-1: ~
Ubuntu-1 console is now available... Press RETURN to get started.
root@Ubuntu-1:~# ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=63 time=61.4 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=63 time=49.2 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=63 time=45.9 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=63 time=38.7 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 38.665/48.777/61.360/8.201 ms
root@Ubuntu-1:~# ping -c 4 192.168.20.2
PING 192.168.20.2 (192.168.20.2) 56(84) bytes of data.
64 bytes from 192.168.20.2: icmp_seq=1 ttl=63 time=56.7 ms
64 bytes from 192.168.20.2: icmp_seq=2 ttl=63 time=65.6 ms
64 bytes from 192.168.20.2: icmp_seq=3 ttl=63 time=42.3 ms
64 bytes from 192.168.20.2: icmp_seq=4 ttl=63 time=194 ms

--- 192.168.20.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3022ms
rtt min/avg/max/mdev = 42.317/89.571/193.611/60.641 ms
root@Ubuntu-1:~#
```

Рисунок 4.3 — Ubuntu



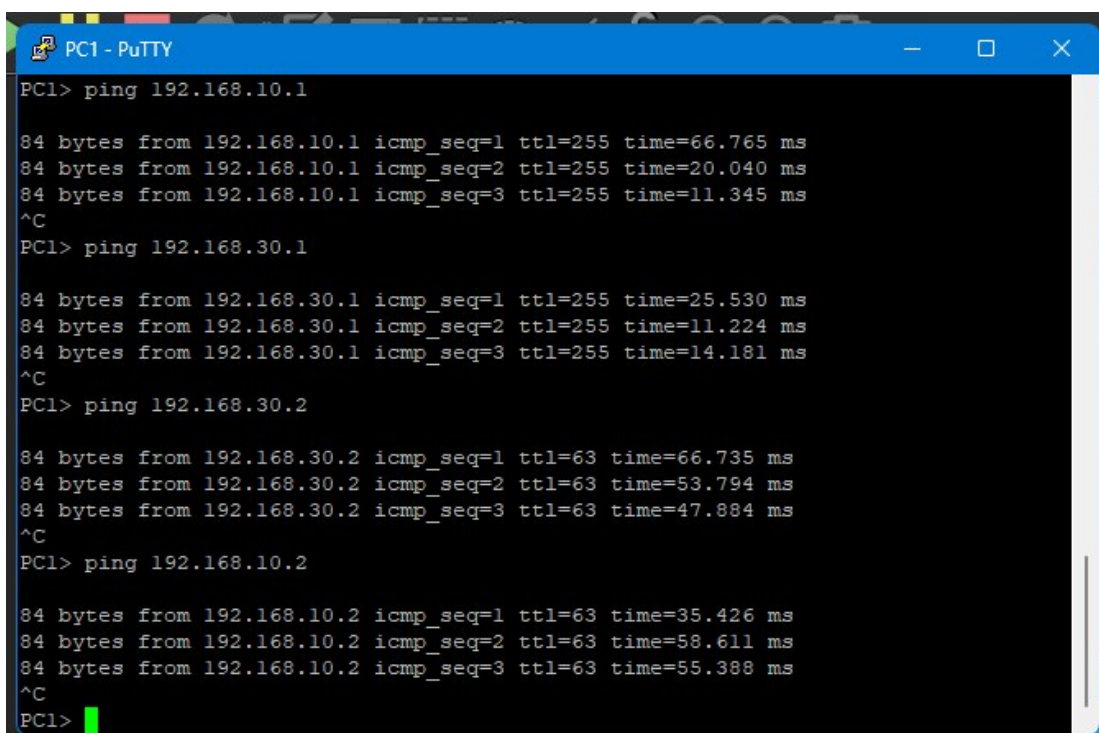
```
Help:      https://sonic-net.github.io/SONiC/

admin@sonic:~$ ping -c 4 192.168.20.2
PING 192.168.20.2 (192.168.20.2) 56(84) bytes of data.
64 bytes from 192.168.20.2: icmp_seq=1 ttl=63 time=98.5 ms
64 bytes from 192.168.20.2: icmp_seq=2 ttl=63 time=497 ms
64 bytes from 192.168.20.2: icmp_seq=3 ttl=63 time=59.0 ms
64 bytes from 192.168.20.2: icmp_seq=4 ttl=63 time=69.4 ms

--- 192.168.20.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3222ms
rtt min/avg/max/mdev = 59.027/180.965/496.925/182.993 ms
admin@sonic:~$ ping -c 4 192.168.30.2
PING 192.168.30.2 (192.168.30.2) 56(84) bytes of data.
64 bytes from 192.168.30.2: icmp_seq=1 ttl=63 time=58.6 ms
64 bytes from 192.168.30.2: icmp_seq=2 ttl=63 time=210 ms
64 bytes from 192.168.30.2: icmp_seq=3 ttl=63 time=39.0 ms
64 bytes from 192.168.30.2: icmp_seq=4 ttl=63 time=68.4 ms

--- 192.168.30.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 39.039/94.134/210.420/67.967 ms
admin@sonic:~$
```

Рисунок 4.4 — SONiC



```
PC1> ping 192.168.10.1

84 bytes from 192.168.10.1 icmp_seq=1 ttl=255 time=66.765 ms
84 bytes from 192.168.10.1 icmp_seq=2 ttl=255 time=20.040 ms
84 bytes from 192.168.10.1 icmp_seq=3 ttl=255 time=11.345 ms
^C
PC1> ping 192.168.30.1

84 bytes from 192.168.30.1 icmp_seq=1 ttl=255 time=25.530 ms
84 bytes from 192.168.30.1 icmp_seq=2 ttl=255 time=11.224 ms
84 bytes from 192.168.30.1 icmp_seq=3 ttl=255 time=14.181 ms
^C
PC1> ping 192.168.30.2

84 bytes from 192.168.30.2 icmp_seq=1 ttl=63 time=66.735 ms
84 bytes from 192.168.30.2 icmp_seq=2 ttl=63 time=53.794 ms
84 bytes from 192.168.30.2 icmp_seq=3 ttl=63 time=47.884 ms
^C
PC1> ping 192.168.10.2

84 bytes from 192.168.10.2 icmp_seq=1 ttl=63 time=35.426 ms
84 bytes from 192.168.10.2 icmp_seq=2 ttl=63 time=58.611 ms
84 bytes from 192.168.10.2 icmp_seq=3 ttl=63 time=55.388 ms
^C
PC1>
```

Рисунок 4.5 — VPCS

5 Заключение

В ходе лабораторной работы были установлены и настроены GNS3 и GNS3 VM в VirtualBox. В проект добавлены и соединены маршрутизатор Cisco 7200, виртуальный коммутатор SONiC VS, контейнер Ubuntu Docker и хост VPCS. Для узлов настроены IP-адреса и шлюзы, после чего проверена связность между ними с помощью команды ping. В результате была подготовлена тестовая сетевая топология и освоены основные приёмы работы с GNS3.